Table of Contents

Introduction **2**

Why we need API service layer **3**

Change code and use ItemService.tsx **4**

Create ItemService.tsx *4*

Change ItemListPage.tsx *5*

Change ItemCreatePage.tsx *7*

Change ItemUpdatePage.tsx: *8*

Demo: API Service Layer

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT/Claude.

Introduction

This demo will guide you through the design pattern of API service layer for managing HTTP requests and data access in a React application. It covers these topics:

Why We Need an API Service Layer

- Separation of Concerns
- Reusability
- Testability
- Error Handling
- Improved Readability
- Future Scalability

Change code and Use of ItemService.tsx

- Created a new file ItemService.tsx to centrally manage HTTP requests.
- Changes in `ItemListPage.tsx`: the `useQuery` call's `queryFn` is set directly to `ItemService.fetchItems`, and the delete mutation's `mutationFn` is set to `ItemService.deleteItem`.
- Changes in ItemCreatePage.tsx: Use ItemService.createItem for HTTP POST
- Changes in ItemUpdatePage.tsx: Use ItemService.fetchItemById to Get item by ID, Use ItemService.updateItem for HTTP PUT requests.

Why we need API service layer

You may have noticed that we have a lot of Http requests to the server, scattered in different pages of items. It makes sense to centrally manage them. The design pattern of isolating calls into a separate service file is commonly referred to as the **"API Service Layer"** pattern or **"Data Access Layer"** (DAL). To distinguish from the DAL in the backend, we refer to this design pattern as API Service Layer.

API Service Layer: This design pattern defines a set of services that encapsulate the business logic and data access logic of an application. It provides a centralised place for all interactions with external data sources, such as APIs or databases.

The design pattern offers several benefits:

1. Separation of Concerns

- **Encapsulation**: Keeps the data fetching logic separate from the UI logic. This separation makes it clear where data fetching is managed and where UI updates occur.
- **Maintainability**: Changes to data fetching (e.g., URL changes, request parameters) are confined to one file, making maintenance easier.

2. Reusability

- **Centralised Service**: By placing all related HTTP requests in a single service file, you can easily reuse these functions across different components without duplicating code.
- **Consistent API**: Ensures that all components use the same methods and handle errors in a consistent manner.

3. Testability

- **Easier Testing**: Isolated services can be tested independently from UI components. This makes unit testing simpler and more focused.
- **Mocking**: Easier to mock HTTP requests during testing since the requests are centralied in one location.

4. Error Handling

- **Centralised Error Handling:** The logic for detecting a failed request and throwing a consistent error (via `handleResponse`) lives in one place, so every page throws errors in the same shape rather than each implementing its own status check.

5. Improved Readability

- **Cleaner Components**: Components are more readable and focused on rendering and user interactions rather than managing data fetching.
- **Focused Service**: The service file focuses solely on data retrieval and manipulation, which improves readability and organization.

6. Future Scalability

- **Modular Growth**: As the application grows, additional methods or modifications to existing ones can be managed easily without affecting component code.
- **Flexible Adjustments**: Changes in API structure or endpoints can be handled in the service file without requiring changes in multiple components.

Change code and use ItemService.tsx

Create ItemService.tsx

We create a new file ItemService.tsx under src/items/. This centralises every Http request the app makes, so `ItemListPage`, `ItemCreatePage`, and `ItemUpdatePage` no longer talk to `fetch` directly at all. They only call functions like `ItemService.createItem(item)`.



The `headers` constant and `handleResponse` function at the top of the file are shared helpers, reused by every request below them, so the "check response, parse JSON" logic is written once rather than repeated in every function. `handleResponse` is written generically (`handleResponse<T>`), so the same function can return a single `Item`, an `Item[]`, or (in principle) any other shape, depending on what each caller needs. This is why `fetchItems` and `fetchItemById` both use it despite returning different types.

Each exported function below wraps one specific Http request: `fetchItems` (Get, the item list), `fetchItemById` (Get, a single item by id), `createItem` (Post), `updateItem` (Put), and `deleteItem` (Delete). Note `deleteItem` does **not** use `handleResponse`, because a successful Delete response typically has no JSON body. Calling `.json()` on it would throw an error, so it only checks `response.ok` directly.

Also note: `fetchItemById` takes `itemId` as a `string`, since it's extracted from the URL by `useParams` in `ItemUpdatePage.tsx`, and URL parameters are always strings. Every other function that needs an item id (`updateItem`, `deleteItem`) receives it as a `number`, taken from `item.itemId` on an already-typed `Item` object.

We now also unify all the http-calling functions, including the one previously in `ItemListPage.tsx`, to plain `async function` declarations rather than a mix of `const ... = async () => {}` and `async function() {}`. Both forms are functionally identical and equally common in modern codebases, but using one consistently avoids the reader wondering whether the different syntax signals a different behaviour.

Change ItemListPage.tsx

We change the ItemListPage.tsx in 3 places.

- `import * as ItemService from './ItemService';` imports every exported member of the `ItemService` module as a single object named `ItemService`, so each function is accessed as `ItemService.fetchItems`, `ItemService.deleteItem`, and so on. This is a convenience import; you could instead import only the specific functions you need (e.g. `import { fetchItems, deleteItem } from './ItemService'`). Either is valid, but importing the whole module as a namespace makes it visually obvious, at every call site, that the function is a network request coming from the service layer.
- The `useQuery` call's `queryFn` now uses `ItemService.fetchItems` directly for the Http Get request, replacing the local `fetchItems` function that previously lived in this file.
- The delete mutation's `mutationFn` now uses `ItemService.deleteItem` for the Http Delete request.
- All standalone code for http calls to servers are deleted.

ItemListPage.tsx U

6-React-Intro > Demo-react-8-itemservice > myshop > src > items > ItemListPage.tsx > ...

```tsx
import { useState, useEffect } from 'react';
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { Button, Form } from 'react-bootstrap';
import ItemTable from './ItemTable';
import ItemGrid from './ItemGrid';
import { Item } from '../types/item';
import { Link } from 'react-router-dom';
import * as ItemService from './ItemService';

const API_URL = import.meta.env.VITE_API_URL;

export default function ItemListPage () {
  const [showTable, setShowTable] = useState<boolean>(() => {
  const savedViewMode = localStorage.getItem('itemViewMode');
  return savedViewMode !== 'grid'; // defaults to table view if nothing saved
});

  useEffect(() => {
  localStorage.setItem('itemViewMode', showTable ? 'table' : 'grid');
}, [showTable]);

  const [searchQuery, setSearchQuery] = useState<string>('');

  const toggleTableOrGrid = () => setShowTable(prev => !prev);

  const { data: items = [], isLoading, error, refetch } = useQuery<Item[], Error>({
    queryKey: ['items'],
    queryFn: ItemService.fetchItems,
  });

  const queryClient = useQueryClient();

  const deleteMutation = useMutation({
    mutationFn: ItemService.deleteItem,
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['items'] });
    },
  });

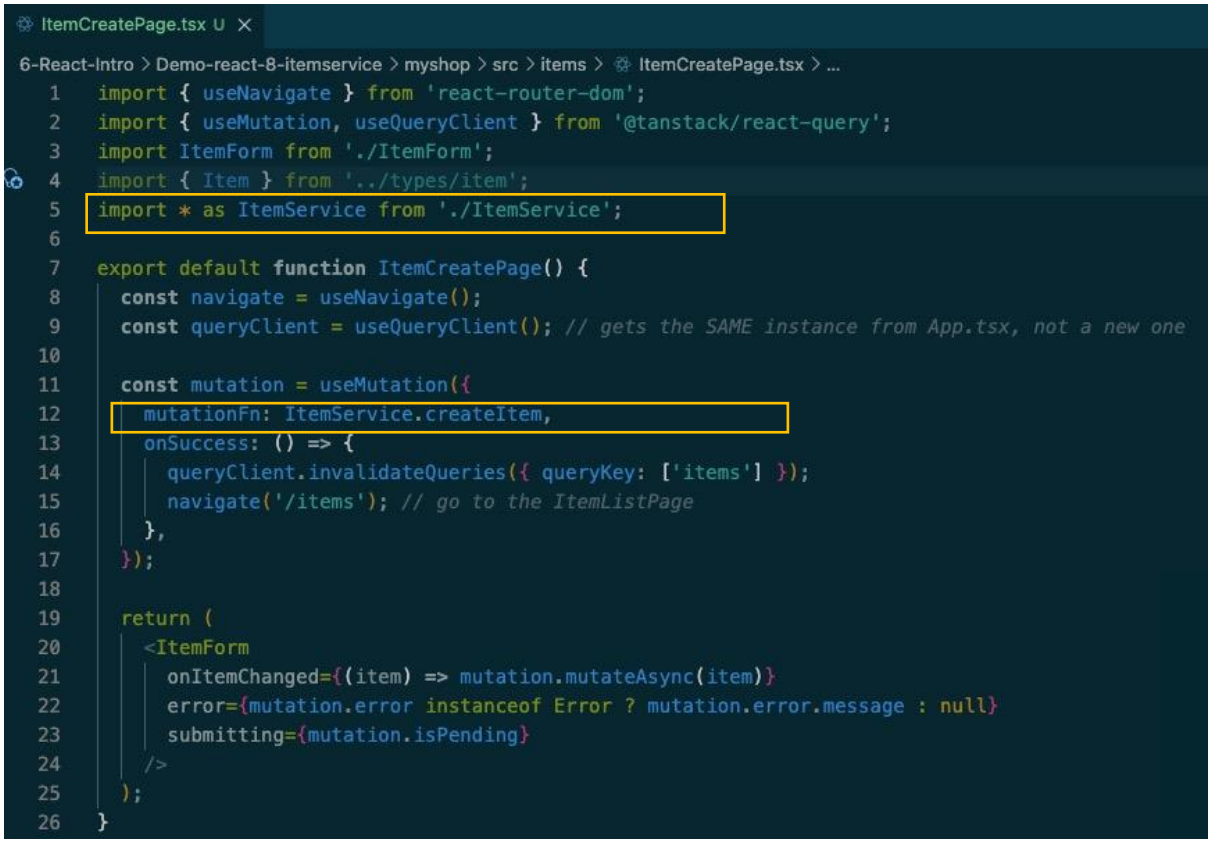
  const handleItemDeleted = (itemId: number) => {
    const confirmDelete = window.confirm(`Are you sure you want to delete the item ${itemId}?`);
    if (confirmDelete) {
      deleteMutation.mutate(itemId);
    }
  };

  const filteredItems = items.filter(item =>
    item.name.toLowerCase().includes(searchQuery.toLowerCase()) ||
    (item.description ?? '').toLowerCase().includes(searchQuery.toLowerCase())
  );

  return (…
  );
};
```

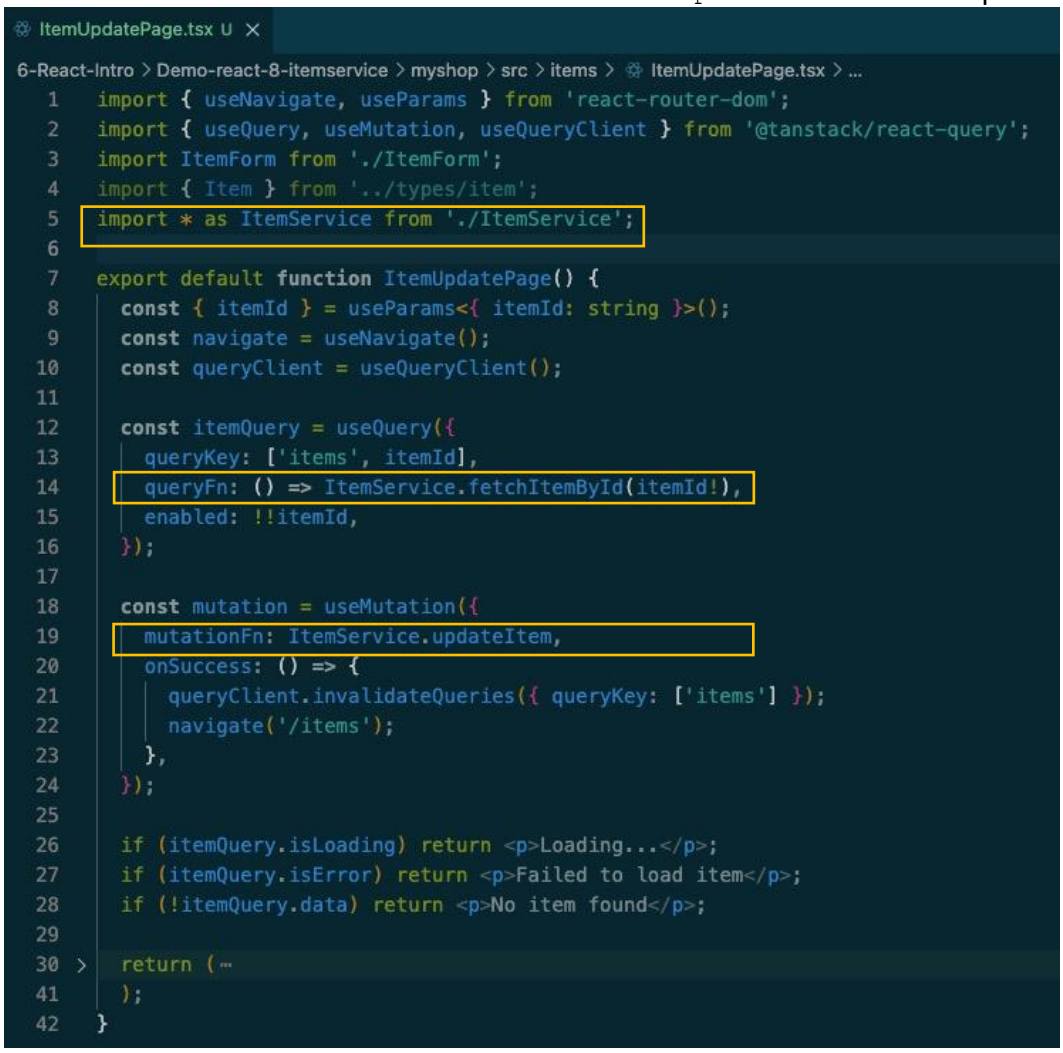
Change ItemCreatePage.tsx

- Imports all functions from `ItemService`.
- The mutation's `mutationFn` uses `ItemService.createItem` for the Http Post request, replacing the local `createItem` function.



Change ItemUpdatePage.tsx:

- Imports all functions from `ItemService`.
- The item query's `queryFn` uses `ItemService.fetchItemById` for the Http Get-by-id request.
- The mutation's `mutationFn` uses `ItemService.updateItem` for the Http Put request.



Now all Http requests are centrally managed in `ItemService.tsx`, and the rest of the code looks much cleaner. Each page focuses purely on *when* to call a request and *what to do with the result*, not *how* the request is made. If the backend's API endpoints ever change, only `ItemService.tsx` needs to be updated; no other file needs to change. This is an effective example of separation of concerns.

Run the project,

```
cd myshop
npm run dev
```

You will see the exact same results as before.

Congratulation! You have learnt the API service layer design pattern for data access! ☺